

Exam 3

● Graded

Student

Cayden Gosseck

Total Points

84 / 100 pts

Question 1

Vocabulary/Theory

3 / 3 pts

✓ + 3 pts Correct

Question 2

Vocabulary/Theory

3 / 3 pts

✓ + 3 pts Correct

Question 3

Vocabulary/Theory

0 / 3 pts

✓ + 0 pts Incorrect

Question 4

Vocabulary/Theory

3 / 3 pts

✓ + 3 pts Correct

Question 5

Vocabulary/Theory

0 / 3 pts

✓ + 0 pts Incorrect

Question 6

Vocabulary/Theory

3 / 3 pts

✓ + 3 pts Correct

Question 7

Vocabulary/Theory

4 / 4 pts

7.1 — Vocabulary/Theory

1.5 / 1.5 pts

✓ - 0 pts Correct

7.2 — Vocabulary/Theory

2.5 / 2.5 pts

✓ + 2.5 pts Correct

Question 8

JavaFX

4 / 4 pts

8.1 JavaFX

2 / 2 pts

✓ - 0 pts Correct

8.2 JavaFX

2 / 2 pts

✓ + 2 pts Correct

Question 9

Vocabulary/Theory

4 / 4 pts

✓ - 0 pts Correct

Question 10

Vocabulary/Theory

3 / 3 pts

✓ + 3 pts Correct

Question 11

Java Code

27.5 / 30 pts

11.1 Java Code

15 / 17 pts

levelUp() method

✓ - 2 pts Does not update existing level or updates incorrectly

11.2 Java Code

12.5 / 13 pts

Default constructor

✓ - 0.5 pts No parameter to this method

Question 12

Vocabulary/Theory

3 / 3 pts

✓ + 3 pts Correct

Question 13

Vocabulary/Theory

0 / 3 pts

✓ + 0 pts Incorrect

Question 14

Vocabulary/Theory

3 / 3 pts

✓ - 0 pts Correct

Question 15

Reading UML

20.5 / 25 pts

15.1 Reading UML/Java Code

4 / 4 pts

✓ - 0 pts Correct

15.2 Reading UML/Java Code

3.5 / 4 pts

✓ - 0.5 pts `interface` should be lowercase

15.3 Reading UML/Java Code

5 / 5 pts

✓ - 0 pts Correct

15.4 Reading UML

2 / 2 pts

✓ + 2 pts Correct

15.5 Reading UML

2 / 2 pts

✓ + 2 pts Correct

15.6 Reading UML

2 / 2 pts

✓ + 2 pts Correct

15.7 Reading UML

2 / 2 pts

✓ + 2 pts Correct

15.8 Reading UML

0 / 4 pts

✓ - 4 pts Incorrect

Question 16

Vocabulary/Theory

3 / 3 pts

✓ + 3 pts Correct

Q1 Vocabulary/Theory

3 Points

How are JavaFX GUI applications visually structured/arranged?

- ☒ The window is a Stage, which holds a Scene, which holds Nodes.
- ☐ The window is a Scene, which holds a Stage, which holds Nodes.
- ☐ The window is a Node, which holds a Stage, which holds Scenes.
- ☐ The window is a Stage, which holds a Node, which holds Scenes.

Q2 Vocabulary/Theory

3 Points

In Java code, how can a class indicate that it is implementing multiple interfaces?

- ☒ Using a comma list of interfaces after the `implements` keyword
- ☐ Using a comma list of interfaces before the `implements` keywords
- ☐ Using multiple `implements` keywords before any `extends` keywords
- ☐ Using multiple `implements` keywords after any `extends` keywords

Q3 Vocabulary/Theory

3 Points

What is the connection between abstract classes/interfaces and polymorphism, if any?

- ☐ Both can be used as the data type for any class extending/implementing them
- ☐ Interfaces can only be used by classes that have a strong relationship to the interface
- ☐ Both exhibit the “many forms” behavior since many classes can use them
- ☒ Abstract classes can be used to represent many different types of classes

Q4 Vocabulary/Theory

3 Points

What makes a marker interface (for example `Cloneable` or `Serializable`) uniquely different from non-marker interfaces?

- ☐ Clones object instances
- ☐ Gives the class a usable name
- ☐ Represents a characteristic of the class
- ☒ Empty interface that contains no methods

Q5 Vocabulary/Theory

3 Points

What is the purpose of an abstract class in OOP?

- ☒ To define classes that the user cannot instantiate
- ☐ To create a blank template class
- ☐ To provide commonality between related classes
- ☐ To provide commonality between related or unrelated classes

Q6 Vocabulary/Theory

3 Points

What is the role of the event handler in event-driven programming?

- ☐ To hold information about events
- ☐ To delegate responsibilities to different objects
- ☐ To generate an Event object upon an action
- ☒ To handle and respond to generated events

Q7 Vocabulary/Theory

4 Points

Answer the following questions.

Q7.1 Vocabulary/Theory

1.5 Points

What data type should the `Comparable` interface's `compareTo()` method return?

- ☐ `Comparable`
- ☐ `boolean`
- ☒ `int`
- ☐ `Object`

Q7.2 Vocabulary/Theory

2.5 Points

How should the `Comparable` interface's `compareTo()` method's return value be determined?

- ☐ `compareTo()` can only compare `Comparable` objects, so an instance of `Comparable` must be returned
- ☐ `true` if the passed object is equal to the calling object, `false` if not
- ☐ Whichever `Object` is larger should be returned, if equal, either `Object` can be returned
- ☒ 0 if the objects are equal, a positive int if the calling object is "larger", a negative int if the calling object is "smaller"

Q8 JavaFX

4 Points



Use the above screenshot for the next set of questions.

Q8.1 JavaFX

2 Points

Based on the provided code, explain the actions that will happen when `btn` is clicked:

Btn's text will be set to "clicked" and lbl's text will be set to "I was ".

Q8.2 JavaFX

2 Points

Based on the provided code, what event-handling technique is being used?

- ☐ Named inner classes with instances created for each event handler
- ☐ Lambda expression handlers
- ☒ Anonymous inner class handlers
- ☐ Manual event handlers

Q9 Vocabulary/Theory

4 Points

Why do lambda expressions only work with Single Abstract Method (SAM)/functional interfaces? Choose the most correct answer.

- ☐ Functional interfaces have nameless methods so developers cannot specify a name
- ☐ All JavaFX interfaces are required to be functional and must have no implementation
- ☐ The JVM needs to know the name of a method before it can execute any code
- ☒ Type inference for method parameters requires only a single possible method match

Q10 Vocabulary/Theory

3 Points

What is the role of the event source in event-driven programming?

- ☐ To hold information about events
- ☒ To generate an Event object upon an action
- ☐ To delegate responsibilities to different objects
- ☐ To handle and respond to generated events

Q11 Java Code

30 Points



Use the above diagram for the next set of questions.

Q11.1 Java Code

17 Points

Given the provided UML, implement the `RPGCharacter` class in Java.

Implement **only** the class header, constructor(s), and method(s) for the class as represented in the UML diagram and listed below:

- Implement the class header with all relevant information.
- Implement the listed convenience constructor.
- Implement the `useSpecialSkill()` method. The method is italicized in the UML.
- Implement the `levelUp()` method in `RPGCharacter` so that the character's `charLevel` is raised or lowered by the amount of levels specified in the parameter. The maximum `charLevel` a character may have is 100, negative levels do not exist, and the method should enforce these rules. Additionally, the character's `healthPool` should increase or decrease by 10% for every `charLevel` gained (for instance, +5 levels = 50% increase in health). Finally, the method should return the character's new `charLevel` after this process.

For all items, use best practices according to Java/OOP convention.

```
public abstract class RPGCharacter implements Levelable {
```

```
    // data
    private String name;
    private String type;
    private double healthPool;
    private int charLevel;
    private String specialSkill;
```

```
    protected RPGCharacter(String name, String type) {
        setName(name);
        setType(type);
    }
```

```
    @Override
    public int levelUp(int numLevels) {
        int currentCharLevel = charLevel;
        int newCharLevel = charLevel + numLevels;
```

```
        // entered 0 for numLevels
        if(currentCharLevel == newCharLevel) {
            return newCharLevel;
        }
```

```
        // level above 100
        if(newCharLevel > 100) {
```

```

        System.out.println("Cannot have above 100 level");
        newCharLevel = 100;
        setCharLevel(newCharLevel);
        setHealthPool(getHealthPool() + (getHealthPool() *
(newCharLevel * .10)));
        return newCharLevel;
    }

    // level is negative
    if(newCharLevel < 0) {
        System.out.println("Cannot have negative levels");

        newCharLevel = 0;
        setCharLevel(newCharLevel);
        setHealthPool(getHealthPool() + (getHealthPool() *
(newCharLevel * .10)));
        return newCharLevel;
    }

    // leveled down
    if(newCharLevel < currentCharLevel) {
        setCharLevel(newCharLevel);
        setHealthPool(getHealthPool() - (getHealthPool() *
(newCharLevel * .10)));
        return newCharLevel;
    }

    setHealthPool(getHealthPool() + (getHealthPool() * (newCharLevel *
.10)));
    setCharLevel(newCharLevel);

    return newCharLevel;
}

// getters and setters
public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

```

```
public String getType() {  
    return type;  
}  
  
public void setType(String type) {  
    this.type = type;  
}  
  
public double getHealthPool() {  
    return healthPool;  
}  
  
public void setHealthPool(double healthPool) {  
    this.healthPool = healthPool;  
}  
  
public int getCharLevel() {  
    return charLevel;  
}  
  
public void setCharLevel(int charLevel) {  
    this.charLevel = charLevel;  
}  
  
public String getSpecialSkill() {  
    return specialSkill;  
}  
  
public void setSpecialSkill(String specialSkill) {  
    this.specialSkill = specialSkill;  
}  
  
public abstract void useSpecialSkill();  
}
```

Q11.2 Java Code

13 Points

Given the provided UML, implement the `Warrior` class in Java.

Implement **only** the class header, constructor(s), and method(s) for the class as represented in the UML diagram as listed below:

- Implement the class header with all relevant information.
- Implement the listed default constructor.
- A `Warrior`'s default `rageLevel` should be set to 10 on creation.
- Implement the mutator method for `rageLevel` which should limit the `rageLevel` to 100.
- For the `Warrior`'s `useSpecialSkill()` method, the `rageLevel` should increase by 50 above the current `rageLevel` and should output a message to the Console that indicates that their specific special skill has been activated. This output must include the name of the special skill properly retrieved from the class's data members as well as the `rageLevel` and `healthPool` values.

For all items, use best practices according to Java/OOP convention.

```
class Warrior extends RPGCharacter {
    private int rageLevel;

    public Warrior(String name, String type) {
        super(name, type);
        setRageLevel(10);
    }

    @Override
    public void useSpecialSkill() {
        setRageLevel(getRageLevel() + 50);
        System.out.println("Using " + getSpecialSkill() + "! Current Rage: " +
getRageLevel() + " Current Health: " + getHealthPool());
    }

    public int getRageLevel() {
        return rageLevel;
    }

    public void setRageLevel(int rageLevel) {
        if(rageLevel > 100) {
            this.rageLevel = 100;
            return;
        }
    }
}
```

```
        this.rageLevel = rageLevel;
    }
}
```

Q12 Vocabulary/Theory

3 Points

What major responsibility does a class have when inheriting from an abstract class or implementing an interface?

- ☐ Create new constructors
- ☐ Override all inherited methods
- ☒ Implement all abstract methods
- ☐ Provide accessors and mutators

Q13 Vocabulary/Theory

3 Points

Abstract classes generally have `protected` constructors. What is the purpose of having these constructors marked `protected`?

- ☐ To allow access to users
- ☐ To allow access to subclasses
- ☒ To allow access to all packages
- ☐ To allow access to superclasses

//

Q14 Vocabulary/Theory

3 Points

When cloning objects (using `Cloneable` or manually), which statements are true about shallow and deep copies? Choose all correct answers.

- ☐ Shallow and deep copies are exactly the same, but shallow copies use more resources, so they should never be used
- ☒ A deep copy correctly clones objects with both primitive and reference types
- ☒ A shallow copy correctly clones only objects with primitive types
- ☐ Shallow and deep copies are exactly the same, but deep copies use more resources, so they should never be used

Q15 Reading UML

25 Points



Use the above diagram for the next set of questions.

Q15.1 Reading UML/Java Code

4 Points

Using the simplified UML diagram, provide the Java class header for `LawnEquipment` (include the opening curly brace and visibility)

```
public abstract class LawnEquipment  
{
```

Q15.2 Reading UML/Java Code

4 Points

Using the simplified UML diagram, provide the Java class header for `Repairable` (include the opening curly brace and visibility)

```
public Interface Repairable {
```

Q15.3 Reading UML/Java Code

5 Points

Using the simplified UML diagram, provide the Java class header for `LeafBlower` (include the opening curly brace and visibility)

```
public class LeafBlower extends  
LawnEquipment implements  
Repairable {
```

Q15.4 Reading UML

2 Points

Using the simplified UML diagram, can an instance of `LawnEquipment` be created?

- ☐ Yes
- ☒ No

Q15.5 Reading UML

2 Points

Using the simplified UML diagram, why can instances of `LawnEquipment` be created or not?

- ☐ Concrete classes cannot be instantiated
- ☐ Interfaces cannot be instantiated
- ☐ Interfaces can be instantiated
- ☒ Abstract classes cannot be instantiated
- ☐ Concrete classes can be instantiated
- ☐ Abstract classes can be instantiated

Q15.6 Reading UML

2 Points

Can an instance of `LeafBlower` be created?

- ☐ No
- ☒ Yes

Q15.7 Reading UML

2 Points

Using the simplified UML diagram, why can instances of `LeafBlower` be created or not?

- ☐ Abstract classes can be instantiated
- ☐ Abstract classes cannot be instantiated
- ☐ Interfaces cannot be instantiated
- ☐ Interfaces can be instantiated
- ☐ Concrete classes cannot be instantiated
- ☒ Concrete classes can be instantiated

Q15.8 Reading UML

4 Points

Using the simplified UML diagram, select all data types that could be used to store a collection (Array, ArrayList, etc.) of `LeafBlower`s

☐	<code>extends</code>
☐	<code>LeafBlower</code>
☐	<code>implements</code>
☒	<code>Cloneable</code>
☐	<code>interface</code>
☐	<code>LawnEquipment</code>
☐	<code>class</code>
☐	<code>Object</code>
☐	<code>Repairable</code>
☒	<code>Comparable</code>

Q16 Vocabulary/Theory

3 Points

What is the purpose of an interface in OOP?

- ☐ To create a blank template class
- ☐ To define classes that the user cannot instantiate
- ☐ To provide commonality between related classes
- ☒ To provide commonality between related or unrelated classes